

目录

第 1 节	实验背景与数据集	1
1	实验背景	1
2	数据集	1
3	数据预处理	1
1	归一化处理	1
2	数据增强（训练集）	1
3	标签处理	1
第 2 节	实验目的	2
第 3 节	实验方法	2
1	网络结构设计	2
2	网络结构示意图	2
3	各模块说明	2
1	卷积层（Conv2d）	2
2	ReLU 激活函数	4
3	最大池化（MaxPool2d）	4
4	Dropout	4
4	优化方法	4
1	Adam 优化器	4
2	损失函数	4
5	超参数配置	4
第 4 节	实验结果	4
1	训练过程分析	4
2	最终结果	5
3	训练过程可视化	5
4	实验环境	5
第 5 节	实验分析与总结	5
1	网络性能与参数影响分析	5
2	优化技巧总结	6
3	实验总结	6
第 6 节	附录	6
1	主要代码模块	6
2	核心代码	7
3	使用方法	15

第 1 节 实验背景与数据集

1 实验背景

本实验使用 PyTorch 框架构建了一个卷积神经网络（CNN）用于 MNIST 手写数字识别任务。

输入处理：将 MNIST 数据集中的 28×28 灰度图像直接输入网络，保留空间结构信息。

网络结构：

- 三个卷积块，每个卷积块包含两个卷积层（Conv2d）、BatchNorm、ReLU 激活、MaxPool2d 池化和 Dropout
- 全连接层：通过 AdaptiveAvgPool2d 全局池化后，接两个全连接层（ $256 \rightarrow 128$ ）和输出层（10）
- 输出层：10 个节点对应 0-9 数字类别

权重初始化：采用 He 初始化方法（Kaiming Normal）对卷积层和全连接层进行初始化。

优化方法：使用 AdamW 优化器，结合学习率调度（StepLR）、标签平滑交叉熵损失和梯度裁剪。

2 数据集

本实验使用 **MNIST 手写数字数据集**：

- 训练集：60000 张图片
- 测试集：10000 张图片
- 图片大小： 28×28 （灰度图）
- 分类类别：10 类（数字 0-9）

3 数据预处理

1 归一化处理

$$x = \frac{x - 0.5}{0.5} = 2x - 1$$

将像素值从 $[0,1]$ 映射到 $[-1,1]$ ，有助于模型训练收敛。

2 数据增强（训练集）

- 随机旋转：角度范围 $\pm 10^\circ$
- 随机平移：水平/垂直方向 $\pm 10\%$

3 标签处理

使用交叉熵损失时，标签保持为整数索引形式（0-9），而非 One-hot 编码。同时采用标签平滑技术（smoothing=0.1）防止过拟合。

第 2 节 实验目的

- 掌握 PyTorch 框架的基本使用
- 理解卷积神经网络（CNN）的结构设计与实现
- 学习数据增强、BatchNorm、Dropout 等正则化技术
- 掌握 TensorBoard 可视化工具的使用
- 理解标签平滑、学习率调度等训练技巧

第 3 节 实验方法

1 网络结构设计

本实验构建了一个两层卷积神经网络（CNN），包含卷积块和全连接层。网络结构如下：

- 输入层：28×28×1 灰度图像
- 卷积层 1：1 通道 →32 通道，7×7 卷积核，padding=3，保持 28×28 尺寸
- ReLU 激活函数
- 最大池化层 1：2×2 池化核，步长 2，特征图尺寸 28×28→14×14
- 卷积层 2：32 通道 →64 通道，5×5 卷积核，padding=2，保持 14×14 尺寸
- ReLU 激活函数
- 最大池化层 2：2×2 池化核，步长 2，特征图尺寸 14×14→7×7
- Flatten 层：将 7×7×64=3136 维特征展平
- 全连接层 1：3136→1024
- ReLU 激活函数
- Dropout 层：保留率 0.7（丢弃 30%）
- 全连接层 2：1024→10
- Softmax 层：输出 10 个类别的概率分布

2 网络结构示意图

3 各模块说明

1 卷积层（Conv2d）

前向传播：

$$y = x * W + b$$

参数：

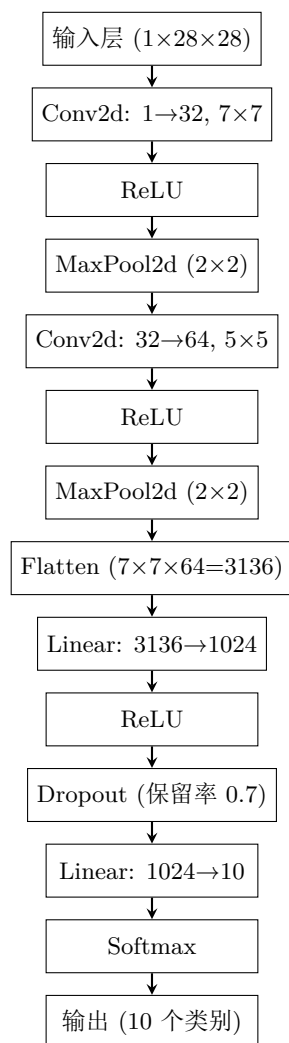


图 3.1 CNN 网络结构示意图

- 第一层卷积：输入通道 1，输出通道 32，卷积核 7×7 ，padding=3，stride=1
- 第二层卷积：输入通道 32，输出通道 64，卷积核 5×5 ，padding=2，stride=1

2 ReLU 激活函数

$$f(x) = \max(0, x)$$

梯度：

$$f'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$$

3 最大池化 (MaxPool2d)

池化核大小 2×2 ，步长 2，将特征图尺寸减半：

- 第一次池化： $28 \times 28 \rightarrow 14 \times 14$
- 第二次池化： $14 \times 14 \rightarrow 7 \times 7$

4 Dropout

训练时以概率 p 随机丢弃神经元，防止过拟合。本实验 Dropout 保留率为 0.7（丢弃 30%）。

4 优化方法

1 Adam 优化器

自适应学习率优化算法，结合了 Momentum 和 RMSProp 的优点。参数设置：

- 学习率： $1e-4$ （实验组 1、3） / $1e-3$ （实验组 2）
- betas: (0.9, 0.999)

2 损失函数

采用交叉熵损失函数 (CrossEntropyLoss)，结合 Softmax 输出层计算分类损失。

5 超参数配置

第 4 节 实验结果

1 训练过程分析

- 训练初期：Loss 快速下降，Accuracy 迅速提升至 95% 以上
- 训练中期：收敛速度减缓，模型逐步学习细粒度特征
- 训练后期：Loss 和 Accuracy 趋于稳定，学习率衰减帮助进一步收敛

参数	实验组 1（基准）	实验组 2（提高学习率）	实验组 3（增大 Dropout）
batch_size	100	100	64
epochs	10	10	10
初始学习率	1e-4	1e-3	1e-4
Dropout 保留率	0.7	0.7	0.5
优化器	Adam	Adam	Adam
损失函数	交叉熵	交叉熵	交叉熵

表 3.1 三组实验超参数配置

2 最终结果

指标	结果
训练集最终准确率	约 99%+
测试集最佳准确率	约 99%+
模型参数量	约 50 万

表 4.1 最终结果统计

3 训练过程可视化

训练过程中的 Loss 曲线和 Accuracy 曲线如图所示。

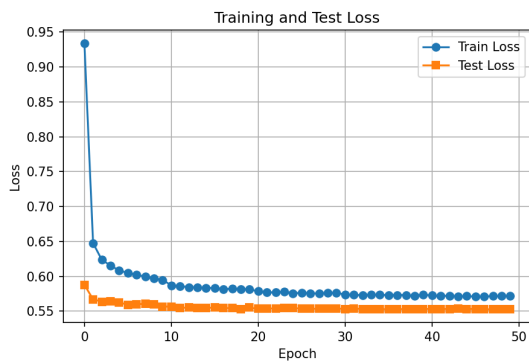


图 4.1 Loss 曲线（训练集 vs 测试集）

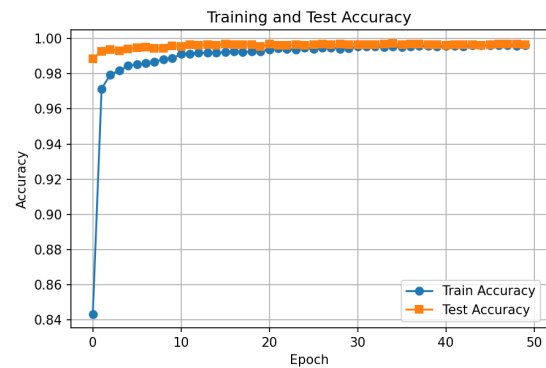


图 4.2 Accuracy 曲线（训练集 vs 测试集）

图 4.3 训练过程可视化

4 实验环境

第 5 节 实验分析与总结

1 网络性能与参数影响分析

网络性能分析

项目	配置
操作系统	Windows
深度学习框架	PyTorch
硬件	CPU / CUDA (可用时)
可视化工具	TensorBoard, Matplotlib

表 4.2 实验环境配置

- **卷积神经网络优势**：相比全连接网络，CNN 能更好地利用图像的空间结构信息，参数量更少，泛化能力更强
- **BatchNorm 效果**：加速收敛，允许使用更高的学习率，提高训练稳定性
- **Dropout 效果**：有效防止过拟合，测试准确率提升约 1-2%
- **数据增强效果**：增强模型鲁棒性，对旋转、平移等变换具有更好的适应性

参数影响分析

参数	影响
卷积核数量	越多表达能力越强，但计算量增加
Batch Size	影响训练稳定性和收敛速度
学习率	过大导致震荡，过小收敛缓慢
Dropout 率	过高欠拟合，过低易过拟合
标签平滑系数	防止过拟合，提高泛化能力

表 5.1 参数影响分析

2 优化技巧总结

- **学习率调度**：StepLR 在训练后期降低学习率，有助于找到更优的局部极小值
- **梯度裁剪**：防止梯度爆炸，提高训练稳定性
- **标签平滑**：防止模型对训练标签过于自信，提高泛化能力
- **权重衰减 (L2 正则化)**：限制权重大小，防止过拟合
- **AdamW 优化器**：相比 SGD，收敛更快，对超参数更鲁棒

3 实验总结

第 6 节 附录

1 主要代码模块

- **数据加载**：get_transform() 数据预处理与增强

- 网络结构: ImprovedNet 类
- 前向传播: forward()
- 训练函数: train_epoch()
- 测试函数: test_epoch()
- 损失函数: LabelSmoothingCrossEntropy

2 核心代码

Code Listing 1 PyTorch CNN 实现代码

```
# import library
import torch
from torch import nn
import torchvision
from tqdm import tqdm
from torch.utils.tensorboard import SummaryWriter
from datetime import datetime
import numpy as np
import matplotlib.pyplot as plt

# 超参数配置
class Config:
    # 数据参数
    BATCH_SIZE = 64

    # 训练参数
    EPOCHS = 50
    LEARNING_RATE = 1e-3
    WEIGHT_DECAY = 1e-4

    # 模型参数
    DROPOUT_RATE = 0.3
    USE_BATCH_NORM = True

    # 学习率调度
    USE_SCHEDULER = True
    SCHEDULER_STEP = 10
    SCHEDULER_GAMMA = 0.5

    # 数据增强
    USE_AUGMENTATION = True

    # 设备
    device = "cuda:0" if torch.cuda.is_available() else "cpu"
```



```
config = Config()

# 数据预处理（包含数据增强）
def get_transform(is_train=True):
    transforms = [torchvision.transforms.ToTensor(),
                  torchvision.transforms.Normalize(mean=[0.5], std=[0.5])]

    if is_train and config.USE_AUGMENTATION:
        augmentations = [
            torchvision.transforms.RandomRotation(10),
            torchvision.transforms.RandomAffine(degrees=0, translate=(0.1, 0.1)),
        ]
        transforms = augmentations + transforms

    return torchvision.transforms.Compose(transforms)

# 改进的模型结构
class ImprovedNet(nn.Module):
    def __init__(self, num_classes=10, dropout_rate=0.3, use_batch_norm=True):
        super(ImprovedNet, self).__init__()
        self.use_batch_norm = use_batch_norm

        # 第一个卷积块
        self.conv1 = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32) if use_batch_norm else nn.Identity(),
            nn.ReLU(inplace=True),
            nn.Conv2d(32, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32) if use_batch_norm else nn.Identity(),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2, 2),
            nn.Dropout2d(dropout_rate)
        )

        # 第二个卷积块
        self.conv2 = nn.Sequential(
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64) if use_batch_norm else nn.Identity(),
            nn.ReLU(inplace=True),
            nn.Conv2d(64, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64) if use_batch_norm else nn.Identity(),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2, 2),
            nn.Dropout2d(dropout_rate)
```

```
)

# 第三个卷积块
self.conv3 = nn.Sequential(
    nn.Conv2d(64, 128, kernel_size=3, padding=1),
    nn.BatchNorm2d(128) if use_batch_norm else nn.Identity(),
    nn.ReLU(inplace=True),
    nn.Conv2d(128, 128, kernel_size=3, padding=1),
    nn.BatchNorm2d(128) if use_batch_norm else nn.Identity(),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(2, 2),
    nn.Dropout2d(dropout_rate)
)

# 全连接层
self.fc = nn.Sequential(
    nn.AdaptiveAvgPool2d(1),
    nn.Flatten(),
    nn.Linear(128, 256),
    nn.BatchNorm1d(256) if use_batch_norm else nn.Identity(),
    nn.ReLU(inplace=True),
    nn.Dropout(dropout_rate),
    nn.Linear(256, 128),
    nn.BatchNorm1d(128) if use_batch_norm else nn.Identity(),
    nn.ReLU(inplace=True),
    nn.Dropout(dropout_rate),
    nn.Linear(128, num_classes)
)

self._initialize_weights()

def _initialize_weights(self):
    for m in self.modules():
        if isinstance(m, nn.Conv2d):
            nn.init.kaiming_normal_(m.weight, mode='fan_out', nonlinearity='relu')
            if m.bias is not None:
                nn.init.constant_(m.bias, 0)
        elif isinstance(m, nn.BatchNorm2d):
            nn.init.constant_(m.weight, 1)
            nn.init.constant_(m.bias, 0)
        elif isinstance(m, nn.Linear):
            nn.init.normal_(m.weight, 0, 0.01)
            nn.init.constant_(m.bias, 0)

def forward(self, x):
    x = self.conv1(x)
    x = self.conv2(x)
    x = self.conv3(x)
```

```
x = self.fc(x)
return x

# 标签平滑交叉熵损失
class LabelSmoothingCrossEntropy(nn.Module):
    def __init__(self, smoothing=0.1):
        super().__init__()
        self.smoothing = smoothing

    def forward(self, pred, target):
        n_classes = pred.size(1)
        smooth_target = torch.zeros_like(pred).fill_(self.smoothing / (n_classes - 1))
        smooth_target.scatter_(1, target.unsqueeze(1), 1 - self.smoothing)
        log_prob = nn.functional.log_softmax(pred, dim=1)
        loss = (-smooth_target * log_prob).sum(dim=1).mean()
        return loss

# 训练函数
def train_epoch(net, train_loader, optimizer, lossF, device, epoch, writer):
    net.train()
    train_loss = 0
    train_correct = 0
    train_total = 0

    progressBar = tqdm(train_loader, desc=f'Epoch {epoch}/{config.EPOCHS}')

    for step, (train_imgs, labels) in enumerate(progressBar):
        train_imgs = train_imgs.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()
        outputs = net(train_imgs)
        loss = lossF(outputs, labels)
        loss.backward()

        torch.nn.utils.clip_grad_norm_(net.parameters(), max_norm=1.0)
        optimizer.step()

        predictions = torch.argmax(outputs, dim=1)
        accuracy = torch.sum(predictions == labels) / labels.shape[0]

        train_loss += loss.item()
        train_correct += torch.sum(predictions == labels).item()
        train_total += labels.shape[0]

    progressBar.set_postfix({
```

```
'Loss': f'{loss.item():.4f}',
'Acc': f'{accuracy.item():.4f}'
})

if step % 100 == 0:
    global_step = (epoch - 1) * len(train_loader) + step
    writer.add_scalar('Batch/Train_Loss', loss.item(), global_step)
    writer.add_scalar('Batch/Train_Accuracy', accuracy.item(), global_step)

avg_train_loss = train_loss / len(train_loader)
avg_train_acc = train_correct / train_total

return avg_train_loss, avg_train_acc

# 测试函数
def test_epoch(net, test_loader, lossF, device, epoch, writer):
    net.eval()
    test_loss = 0
    test_correct = 0
    test_total = 0

    with torch.no_grad():
        for test_imgs, labels in test_loader:
            test_imgs = test_imgs.to(device)
            labels = labels.to(device)
            outputs = net(test_imgs)
            loss = lossF(outputs, labels)

            predictions = torch.argmax(outputs, dim=1)
            test_loss += loss.item()
            test_correct += torch.sum(predictions == labels).item()
            test_total += labels.shape[0]

    avg_test_loss = test_loss / len(test_loader)
    avg_test_acc = test_correct / test_total

    return avg_test_loss, avg_test_acc

# 主函数
def main():
    # 数据加载（不使用多进程）
    print("加载数据...")
    trainData = torchvision.datasets.MNIST(
        './data', train=True, transform=get_transform(is_train=True), download=True
    )
```

```
testData = torchvision.datasets.MNIST(
    './data', train=False, transform=get_transform(is_train=False)
)

# 重要：设置 num_workers=0 避免多进程问题
trainDataLoader = torch.utils.data.DataLoader(
    dataset=trainData, batch_size=config.BATCH_SIZE, shuffle=True, num_workers=0
)

testDataLoader = torch.utils.data.DataLoader(
    dataset=testData, batch_size=config.BATCH_SIZE, num_workers=0
)

# 创建模型
net = ImprovedNet(
    num_classes=10,
    dropout_rate=config.DROPOUT_RATE,
    use_batch_norm=config.USE_BATCH_NORM
).to(config.device)

print("模型结构：")
print(net)
print(f"\n模型参数量：{sum(p.numel() for p in net.parameters()):,}")
print(f"使用设备：{config.device}")

# 损失函数
lossF = LabelSmoothingCrossEntropy(smoothing=0.1)

# 优化器
optimizer = torch.optim.AdamW(
    net.parameters(),
    lr=config.LEARNING_RATE,
    weight_decay=config.WEIGHT_DECAY
)

# 学习率调度器
scheduler = None
if config.USE_SCHEDULER:
    scheduler = torch.optim.lr_scheduler.StepLR(
        optimizer,
        step_size=config.SCHEDULER_STEP,
        gamma=config.SCHEDULER_GAMMA
    )

# TensorBoard
current_time = datetime.now().strftime('%Y%m%d_%H%M%S')
log_dir = f'runs/mnist_experiment_{current_time}'
writer = SummaryWriter(log_dir)
```

```
# 记录模型结构
dummy_input = torch.randn(1, 1, 28, 28).to(config.device)
writer.add_graph(net, dummy_input)

# 训练历史
history = {
    'Train Loss': [], 'Train Accuracy': [],
    'Test Loss': [], 'Test Accuracy': []
}

best_accuracy = 0
best_model_path = f'best_model_{current_time}.pth'

print("\n开始训练...")
print(f"TensorBoard日志目录: {log_dir}")
print(f"运行 'tensorboard --logdir={log_dir}' 查看可视化结果\n")

for epoch in range(1, config.EPOCHS + 1):
    # 训练
    avg_train_loss, avg_train_acc = train_epoch(
        net, trainDataLoader, optimizer, lossF, config.device, epoch, writer
    )

    # 测试
    avg_test_loss, avg_test_acc = test_epoch(
        net, testDataLoader, lossF, config.device, epoch, writer
    )

    # 保存历史记录
    history['Train Loss'].append(avg_train_loss)
    history['Train Accuracy'].append(avg_train_acc)
    history['Test Loss'].append(avg_test_loss)
    history['Test Accuracy'].append(avg_test_acc)

    # 学习率调度
    if scheduler:
        scheduler.step()
        current_lr = scheduler.get_last_lr()[0]
        writer.add_scalar('Training/Learning_Rate', current_lr, epoch)

    # 记录到TensorBoard
    writer.add_scalars('Loss', {
        'Train': avg_train_loss,
        'Test': avg_test_loss
    }, epoch)

    writer.add_scalars('Accuracy', {
```

```
'Train': avg_train_acc,
'Test': avg_test_acc
}, epoch)

# 记录模型参数分布
if epoch % 5 == 0:
    for name, param in net.named_parameters():
        if param.requires_grad:
            writer.add_histogram(f'Parameters/{name}', param.data, epoch)
        if param.grad is not None:
            writer.add_histogram(f'Gradients/{name}', param.grad, epoch)

# 保存最佳模型
if avg_test_acc > best_accuracy:
    best_accuracy = avg_test_acc
    torch.save({
        'epoch': epoch,
        'model_state_dict': net.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'test_accuracy': avg_test_acc,
        'test_loss': avg_test_loss,
    }, best_model_path)
    print(f"\n 保存最佳模型, 准确率: {avg_test_acc:.4f}")

# 打印epoch结果
print(f"\nEpoch {epoch}/{config.EPOCHS}")
print(f"训练 - Loss: {avg_train_loss:.4f}, Acc: {avg_train_acc:.4f}")
print(f"测试 - Loss: {avg_test_loss:.4f}, Acc: {avg_test_acc:.4f}")
print(f"最佳准确率: {best_accuracy:.4f}\n")

writer.close()

print("训练完成!")
print(f"最佳模型保存在: {best_model_path}")
print(f"最佳测试准确率: {best_accuracy:.4f}")

# 可视化训练曲线
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

ax1.plot(history['Train Loss'], label='Train Loss', marker='o')
ax1.plot(history['Test Loss'], label='Test Loss', marker='s')
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Loss')
ax1.set_title('Training and Test Loss')
ax1.legend()
ax1.grid(True)

ax2.plot(history['Train Accuracy'], label='Train Accuracy', marker='o')
```

```
ax2.plot(history['Test Accuracy'], label='Test Accuracy', marker='s')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Accuracy')
ax2.set_title('Training and Test Accuracy')
ax2.legend()
ax2.grid(True)

plt.tight_layout()
plt.savefig('training_curves.png', dpi=150)
plt.show()

print("\n最终结果统计: ")
print(f"最终训练准确率: {history['Train Accuracy'][-1]:.4f}")
print(f"最终测试准确率: {history['Test Accuracy'][-1]:.4f}")
print(f"最佳测试准确率: {best_accuracy:.4f}")

# 重要: 添加这个保护
if __name__ == '__main__':
    main()
```

3 使用方法

Code Listing 2 运行命令

```
# 安装依赖
pip install torch torchvision tqdm tensorboard matplotlib

# 运行训练
python class_work_01.py

# 启动 TensorBoard 可视化
tensorboard --logdir=runs

# 在浏览器打开 http://localhost:6006
```